

Distribution — A Readable Companion to the OTP Docs

Chapter 6 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why distribution looks easy and isn't

Distributed Erlang makes a remote process look like a local one: the same pids, the same `!`, the same links and monitors. That transparency is the attraction and the trap. The syntax hides the network, but the network still fails in ways that one machine never does.

On one node, a monitored process that stops answering is dead. Across nodes, it might be dead, slow, or cut off by a partition while still running happily on the other side. Every design choice in this chapter comes back to that ambiguity: from inside one node, *unreachable* looks exactly like *dead*.

2. The building blocks

Concept	What it is	Watch out for
Node name	<code>name@host</code> , set with <code>-sname</code> (short) or <code>-name</code> (long)	Short-name and long-name nodes cannot talk to each other
Cookie	Shared secret checked at connection time	Protects against accidents, not attackers
epmd	Local daemon mapping node names to ports	Must be reachable, and must be firewalled
Transitive connect	Connecting to B also connects you to B's peers	On by default; <code>-connect_all false</code> turns it off
Hidden node	Connects without joining the mesh	Not listed by <code>nodes/0</code> ; good for tooling and consoles
<code>global</code>	Cluster-wide name registry and locks	Name clashes after a partition are resolved by killing one process

Security, in the docs' own words

The cookie is not encryption. The docs are blunt: starting a distributed node without `-proto_dist inet_tls` exposes it “to attacks that may give the attacker complete access to the node and by extension the cluster.” Traffic is in clear text by default. Any node that connects can run any code on yours.

Detecting failure

Nodes exchange ticks. With the default `net_ticktime` of 60 seconds, a silent peer is declared down after 45 to 75 seconds. Then every link to a process on that node fires with reason `noconnection`, and every

monitor delivers 'DOWN' with `noconnection.net_kernel:monitor_nodes/1` lets a process watch nodes come and go.

3. How the runtime actually does it

One connection per pair of nodes

All traffic between two nodes shares one connection: messages, links, monitors, ticks. Ordering between one sender and one receiver still holds, as in chapter 2. But a large message sits in the same pipe as everything else.

The distribution buffer

Outgoing data queues in a per-connection buffer. When it passes the busy limit (`+zdbb1`, default 1024 KB), every process sending to that node is suspended until the buffer drains. One slow peer or one huge message can freeze senders all over your node.

Signals can be lost

The docs are explicit: “signals can be lost if the distribution channel goes down.” A successful `!` means the message was handed to the runtime, not that it arrived. When the connection drops, whatever was in flight is gone, and the only notice you get is `noconnection` on links and monitors.

Partitions and global

When connections break, `global` (since OTP 25, by default) actively disconnects nodes to avoid overlapping partitions, so the cluster splits into clean, fully connected groups. When groups merge and two of them registered the same global name, the default resolver picks one pid at random and kills the other.

4. Plausible but wrong

4.1 “Node down” means “process dead”

```
handle_info({'DOWN', _, process, _Pid, noconnection}, State) ->
    promote_self_to_leader(),
    {noreply, State}.
```

`noconnection` means the link failed, not that the remote process stopped. During a partition, both sides see each other as down and both promote themselves: two leaders, two writers, divergent data. Fix: treat `noconnection` as “unknown”. For anything that must be single-leader, use a consensus library (such as `ra`, which implements Raft) or an external coordinator, not node monitoring.

4.2 Treating a send as a delivery

```
replicate(Nodes, Update) ->
    [{store, N} ! {apply, Update} || N <- Nodes],
    ok.
```

This returns `ok` whether or not any replica got the update. A connection that drops mid-send silently loses it. Fix: have replicas acknowledge, retry on missing acks, and make updates idempotent (version numbers or unique ids) so retries are safe.

4.3 Shipping a large payload over the cluster link

```
{cache, OtherNode} ! {warm, ets:tab2list(big_cache)}.    %% 500 MB
```

The payload fills the distribution buffer, suspends every other process sending to that node, and delays ticks and control messages behind it. Fix: send in chunks with flow control (the receiver asks for the next chunk), or move bulk data over a separate channel such as a TCP socket or shared storage.

4.4 Sending anonymous funs to other nodes

```
rpc:call(Node, erlang, apply, [fun() -> my_mod:work(Arg) end, []]).
```

A fun is tied to the exact version of the module that created it. If `Node` runs a different build, which is exactly what happens during a rolling upgrade, the call fails with `badfun`. Fix: call by module, function and arguments: `rpc:call(Node, my_mod, work, [Arg])` or `erpc:call(Node, my_mod, work, [Arg])`.

4.5 A cluster on the default cookie

```
erl -name app@10.0.0.5 -setcookie mycookie
```

With ports 4369 (epmd) and the distribution ports open to the network, anyone who guesses or sniffs the cookie can run `os:cmd/1` on your machine. The cookie travels in a challenge that the docs describe as not cryptographically secure. Fix: TLS distribution (`-proto_dist inet_tls`), a firewall that only admits cluster members, and a strong random cookie.

4.6 A global singleton with no plan for merges

```
init([]) ->
    yes = global:register_name(scheduler, self()),
    {ok, load_jobs()}.
```

During a partition each side starts its own `scheduler`. When the network heals, `global` kills one at random, along with any jobs only it knew about. Fix: keep the state that matters outside the process (in a replicated store), make startup reload it, and if needed pass your own resolve function to `global:register_name/3`.

4.7 Remote calls with local expectations

```
handle_call(price, _From, State) ->  
    P = gen_server:call({pricing, remote@host}, get),  
    {reply, P, State}.
```

When the remote node hangs rather than crashes, this blocks for the full 5-second call timeout, and it's inside another server's `handle_call`, so that server's callers queue up too. Failure detection itself takes up to 75 seconds. Fix: set explicit short timeouts, avoid remote calls inside callbacks that others wait on, and cache or degrade when the remote side is slow.

5. Review checklist and sources

- ☐ Distribution uses TLS, and `epmd` plus the distribution ports are firewalled
- ☐ `noconnection` is handled as “unknown”, not “dead”
- ☐ No single-leader logic depends only on node monitoring
- ☐ Cross-node updates are acknowledged and idempotent
- ☐ Bulk data is chunked or sent outside the distribution channel
- ☐ Remote calls use module/function/arguments, not funs
- ☐ Every global name has a plan for what happens after a partition heals
- ☐ Remote calls have explicit timeouts and don't run inside heavily used callbacks

Sources

- Distributed Erlang, Reference Manual: <https://www.erlang.org/doc/system/distributed.html>
- Processes (signals over distribution), Reference Manual: https://www.erlang.org/doc/system/ref_man_processes.html
- global module reference: <https://www.erlang.org/doc/apps/kernel/global.html>
- Kernel application (`net_ticktime`): https://www.erlang.org/doc/apps/kernel/kernel_app.html
- `erl` command (`+zdbbl`): https://www.erlang.org/doc/apps/erts/erl_cmd.html